

Supabase, PostgREST & Postman

Enhancing PostgreSQL with Auto-Generated APIs & API Testing

September 2025



Supabase: Augmenting PostgreSQL

- Auto-generated REST API via PostgREST: Reflect your schema into REST endpoints without writing code.
- GraphQL API: Query your data using GraphQL queries.
- Realtime subscriptions & presence: Receive database changes over WebSockets and broadcast messages.
- Authentication & Authorization: Email, social, phone and magic link logins with row-level security policies.
- Storage & CDN: S3-compatible file storage with CDN caching and image transformations.
- Edge functions & platform tooling: Globally deployed TypeScript functions, daily backups, branching and network restrictions.





PostgREST: Database as API

- Database as single source of truth: API endpoints mirror tables, views and functions, eliminating duplicate business logic.
- Declarative & performant: Use SQL and PostgreSQL's query planner rather than imperatively looping in code.
- Leak-proof abstraction: Define new endpoints using SQL views with predictable performance characteristics.
- Focused scope: Handles CRUD and authorization; leave routing and caching to other tools.

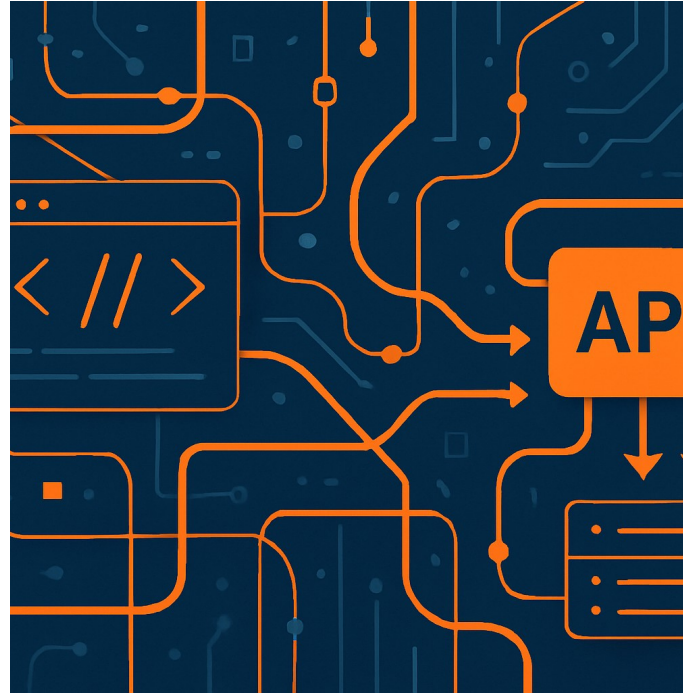


Supabase /rest/v1 API

- Auto-generated & instant: Schema changes are reflected immediately in your API.
- Self-documenting: The dashboard generates documentation for your endpoints.
- Secure & governed: Enforces row-level security and key-based auth via an API gateway.
- Fast & scalable: Single SQL statements yield high throughput; ideal for serverless workloads.
- Comprehensive capabilities: CRUD operations, nested relationships, views, materialized views, foreign tables and functions with roles and grants.

Postman: API Development & Testing

- Send & receive requests: Work with GET, POST, PUT, PATCH and DELETE using query params, headers and body data.
- Authentication helpers: Attach API keys, tokens, OAuth or Basic Auth effortlessly.
- Collections & collaboration: Organize requests into collections, share with teammates and reuse across projects.
- Automated tests & monitors: Write JavaScript tests and schedule monitors to validate your API continuously.
- Docs & mocking: Generate documentation from your collections and spin up mock servers during development.





Testing Supabase APIs with Postman

1. Find your project's API URL (`https://<project_ref>.supabase.co/rest/v1`) and an API key from the Supabase dashboard.
2. Create a new request in Postman and set the URL to an endpoint, e.g., `/users?select=id,name`.
3. Add an ApiKey header with your Supabase key or use the Bearer token helper.
4. Choose the appropriate HTTP method (GET to read, POST to insert, PATCH to update or DELETE to remove).
5. Send the request and inspect the JSON response; add tests and chain requests as needed.

Conclusion

Supabase extends PostgreSQL with realtime updates, authentication, storage, edge functions and auto-generated APIs via PostgREST.

The /rest/v1 endpoint exposes your database schema as a secure, performant REST interface that requires no server code.

Postman empowers you to explore, test and document these APIs through a rich suite of request helpers, collections, monitors and docs.

Together, these tools provide a powerful learning platform for students pursuing software engineering, IT, database administration, project management or cybersecurity roles.

In this course you will practice efficient SQL query strategies, implement row-level security, administer your database and build professional-grade APIs.

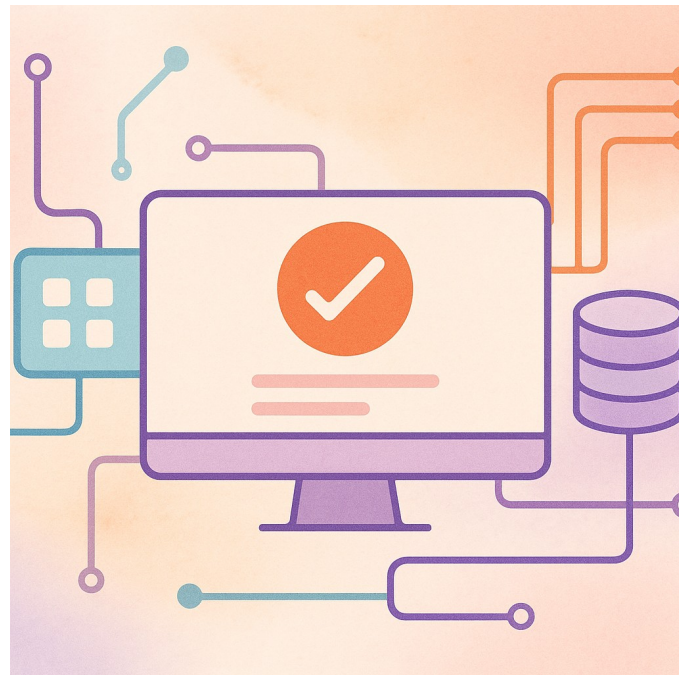
Supabase: Architecture & Features

- Full Postgres database with direct SQL access and extensions.
- Layers services like authentication, storage, realtime and vector search on top of Postgres.
- Database branching & daily backups for safe experimentation.
- Integrated dashboard for management and monitoring.



What Supabase Adds to PostgreSQL

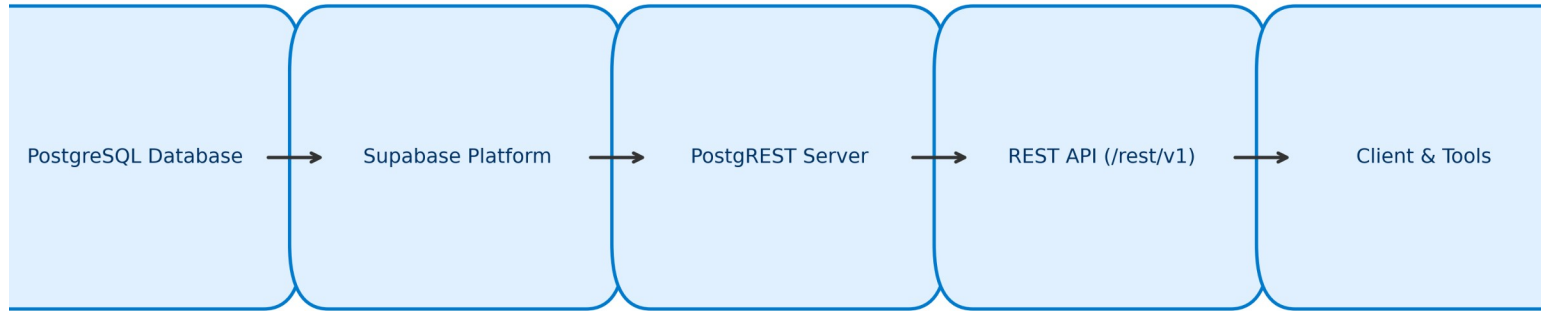
- API layer: REST & GraphQL endpoints vs SQL only.
- Authentication & RLS integrated with user management.
- Realtime streaming over WebSockets.
- Object storage with CDN caching.
- Edge functions & scheduled jobs.
- Vector embeddings & branching for modern workloads.



PostgREST & Declarative Design

- Turns your database directly into a RESTful API; endpoints mirror tables, views and functions.
- Database as single source of truth — no duplicate business logic or ORM layers.
- Declarative programming: rely on SQL and the query planner instead of imperatively looping through records.
- Leak-proof abstraction: create new endpoints by defining views in SQL.
- Focused scope: PostgREST handles CRUD and authorization, letting other tools manage routing and caching.

System Architecture



Data flows from PostgreSQL through the Supabase platform and PostgREST to the /rest/v1 API, which can be consumed by clients like Postman.

Row Level Security & Policies

- Enable RLS on a table: `ALTER TABLE <table> ENABLE ROW LEVEL SECURITY;`
- No data is accessible via the anon key until policies are defined.
- Requests are mapped to Postgres roles: anon and authenticated.
- Policies are SQL expressions evaluated as WHERE clauses.

```
-- Enable RLS and define policies
ALTER TABLE public.todos ENABLE ROW LEVEL SECURITY;
CREATE POLICY todo_select_own ON public.todos FOR SELECT
USING ((auth.uid()) = user_id);
CREATE POLICY todo_modify_own ON public.todos FOR ALL
USING ((auth.uid()) = user_id);
```

From SQL to REST: Examples

SQL

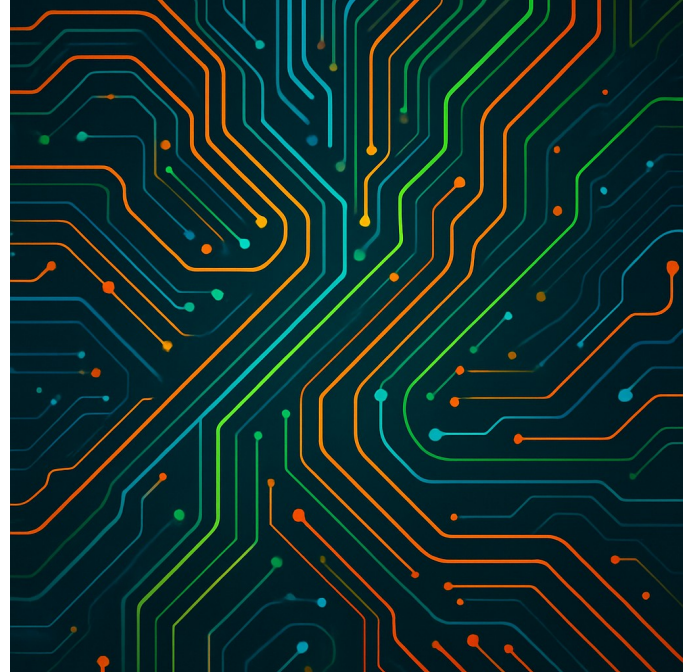
```
SELECT id, task FROM todos WHERE is_done = FALSE ORDER BY
inserted_at DESC;INSERT INTO todos (user_id, task) VALUES
('<uuid>', 'Finish homework');SELECT * FROM
search_todos('homework');
```

REST

```
GET /rest/v1/todos?
select=id,task&is_done=eq.false&order=inserted_at.descPOST
/rest/v1/todos
{ "user_id": "<uuid>", "task": "Finish homework" }
POST /rest/v1/rpc/search_todos
{ "q": "homework" }
```

Postman Features & Workflows

- Collaborate: Create workspaces, comment on requests and share collections.
- Design APIs: Use Spec Hub and API Builder to define your API specification.
- Documentation: Generate and publish API docs from collections.
- Monitoring: Schedule monitors to run tests and track performance.
- Governance & Security: Apply organization-wide rules and security checks.
- Administration & Reports: Manage team access and view usage analytics.



Using Postman with Supabase

- Retrieve your project URL and API keys from the Supabase dashboard.
- Create a Postman environment with variables for `base_url`, `apikey`, `access_token` and `service_role`.
- Construct requests: set URL to `{{base_url}}/rest/v1/<table>` and include the `apikey` and `Authorization` headers.
- Write tests using the `pm` API to validate responses and chain requests.
- Schedule monitors to run your collection on a schedule and receive alerts.